ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Н.В. Путилова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Объектно-реляционные базы данных. Манипуляция данными и пользовательские операторы

по курсу: ПРОЕКТИРОВАНИЕ БАЗ ДАННЫХ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работы:

Разработка и реализация базы данных для системы управления доставкой посылок с использованием наследования таблиц в PostgreSQL.

Описание задания:

1. Выполнить вставку тестовых данных в таблицы, созданные в ходе выполнения лабораторной работы 9.

2. Сделать запросы выборки с условием к таблицам предку и потомку, только потомку и только предку.

3. Придумать и создать пользовательский оператор для своей предметной области

4. Придумать и создать пользовательскую агрегатную функцию для своей предметной области

Вариант 18:

18. служба доставки с учетом разницы физических и юридических лиц: адрес доставки, контактное лицо, стоимость посылки, диапазон желаемого времени доставки. время доставки фактическое, вес посылки, отметка о доставке, фирма отправитель (у юр. лица есть контактное лицо (человек))

а. все посылки, отправляемые на улицу, в названии которой есть «малая», но это не последнее слово в нем

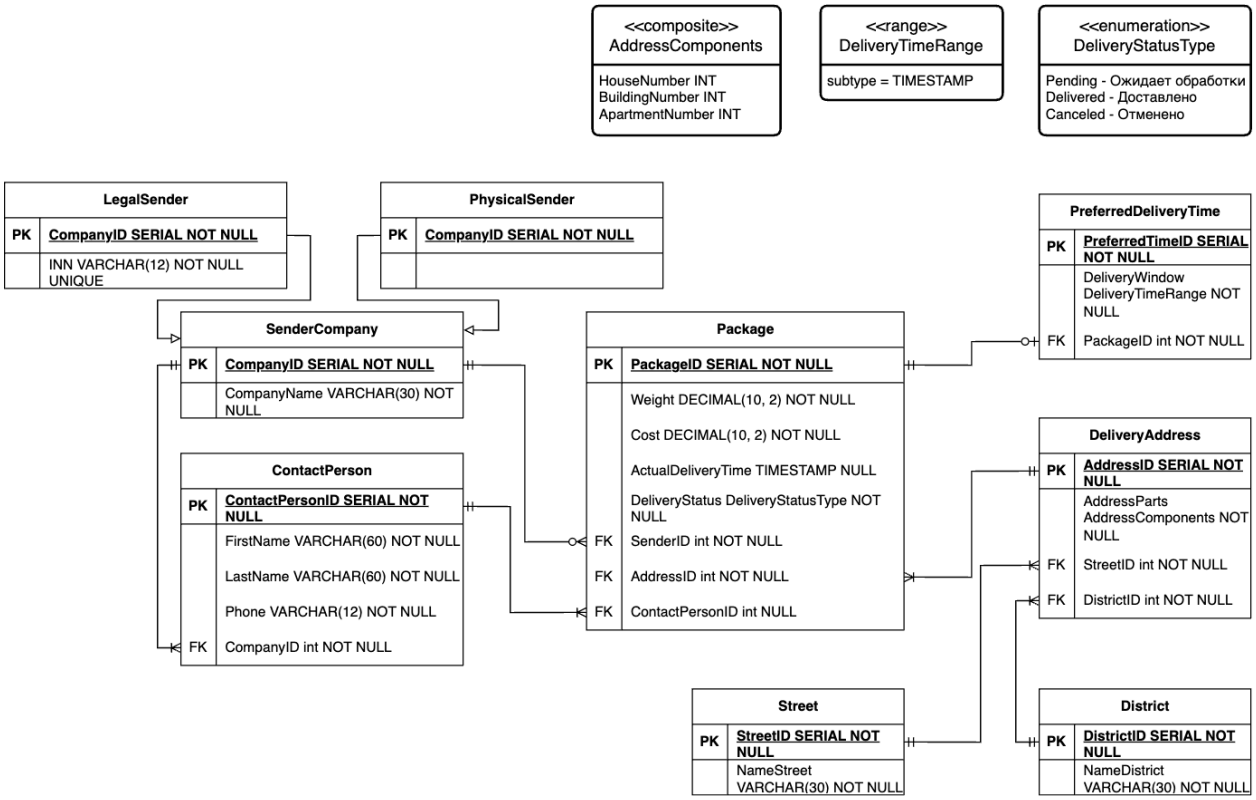
б. контакт(человек) юридического лица, получавшее посылки по разным адресам

в. фирма, отправившая больше среднего посылок для юр. лиц.

г. посылка с самым маленьким весом

д. контактное лицо, не получавшее посылок в 2022 году

Физическая модель БД:



Наборы данных, содержащихся в таблицах БД:

Текст запроса	Данные, удовлетворяющие условиям запроса	Данные, не удовлетворяющие условиям запроса
а. Все посылки, отправляемые на улицу, в названии которой есть «малая», но это не последнее слово в нём	"tracking_id": "ABC123", "tracking_id": "LMN456", "tracking_id": "DEF567", "tracking_id": "LMM455", "tracking_id": "ADD002"	"tracking_id": "XYZ789", "tracking_id": "PQR789", "tracking_id": " ADD001"
б. Контакт (человек) юридического лица, получавшее посылки по разным адресам	"users.contact_person": "Сергей Валерьев", "users.contact_person": "Дмитрий Орлов",	"users.contact_person": "Алексей Смирнов", "users.contact_person": "Ольга Васильевна",
в. Фирма, отправившая больше среднего посылок для юр. лиц	"sender.company": "ООО Альфа"	"sender.company": "ООО Ромашка" "sender.company": "ООО Гамма" "sender.company": "ООО Бета"
г. Посылка с самым маленьким весом	"tracking_id": "ADD001", "parcel_info.weight": 2.5	"tracking_id": "ABC123", "parcel_info.weight": 5.2" "tracking_id": "XYZ789", "parcel_info.weight": 8.7"
д. Контактное лицо, не получавшее посылок в 2022 году	"users.contact_person": "Дмитрий Орлов", "users.contact_person": "Сергей Валерьев", "users.contact_person": "Ольга Васильева"	"recipient.contact_person": Иван Иванов, "recipient.contact_person": null,

Текст запросов на SQL:

Вставка данных из таблицы тестовых данных:

-- 1. Вставляем только в LegalSender (автоматически попадет в SenderCompany)

INSERT INTO LegalSender (CompanyName, INN) VALUES

('ООО Альфа', '123456789012'),

('ООО Ромашка', '987654321098'),

('ООО Гамма', '555555555555'),

('ООО Бета', '666666666666');

-- Добавляем крупную компанию-агрегатор в SenderCompany

INSERT INTO SenderCompany (CompanyName) VALUES

('МегаДоставка Сервис');

-- Добавляем физическое лицо в PhysicalSender

INSERT INTO PhysicalSender (CompanyName) VALUES

('Иванова Мария Сергеевна');

-- 2. Контактные лица

INSERT INTO ContactPerson (FirstName, LastName, Phone, CompanyID) VALUES

('Сергей', 'Валерьев', '79111234567', 1),

('Дмитрий', 'Орлов', '79112234567', 1),

('Алексей', 'Смирнов', '79113234567', 2),

('Ольга', 'Васильевна', '79114234567', 2),

('Ольга', 'Васильева', '79115234567', 3),

('Иван', 'Иванов', '79116234567', 4),

('Мария', 'Иванова', '79117234567', 6); -- Контакт для физического лица (CompanyID=6)

```
-- 3. Улицы
INSERT INTO Street (NameStreet) VALUES
('Малая Дмитровка'),
('Малая Бронная'),
('Большая Малая'),
('Дмитровская');

-- 4. Районы
INSERT INTO District (NameDistrict) VALUES
('Центральный'),
('Северный');

-- 5. Адреса доставки
INSERT INTO DeliveryAddress (AddressParts, StreetID, DistrictID) VALUES
(ROW(10, 2, 401)::AddressComponents, 1, 1),
(ROW(20, 3, NULL)::AddressComponents, 2, 1),
(ROW(10, 0, 2)::AddressComponents, 3, 2),
(ROW(20, 0, NULL)::AddressComponents, 4, 2),
(ROW(10, 0, 10)::AddressComponents, 4, 2),
(ROW(NULL, 0, 0)::AddressComponents, 2, 2),
(ROW(NULL, NULL, NULL)::AddressComponents, 2, 2),
(ROW(20, NULL, 3)::AddressComponents, 2, 2),
(ROW(NULL, NULL, 0)::AddressComponents, 1, 2),
(ROW(NULL, 0, 0)::AddressComponents, 2, 1);

-- 6. Посылки
INSERT INTO Package (Weight, Cost, ActualDeliveryTime, DeliveryStatus, SenderID, AddressID, ContactPersonID) VALUES
-- Сергей Валерьев (ID 1) - нет посылок в 2022
```

```
(5.2, 1000, '2021-06-15 12:00:00', 'Delivered', 1, 1, 1),
(6.0, 1200, '2023-02-20 14:30:00', 'Delivered', 1, 2, 1),

-- Дмитрий Орлов (ID 2) - нет посылок в 2022
(7.0, 1500, '2021-11-30 10:00:00', 'Delivered', 1, 3, 2),
(5.5, 1300, '2023-01-15 09:00:00', 'Delivered', 1, 1, 2),
(4.0, 1100, '2021-12-15 08:00:00', 'Delivered', 1, 2, 2),

-- Алексей Смирнов (ID 3) - есть посылка в 2022
(8.0, 2000, '2022-07-22 13:20:00', 'Delivered', 2, 4, 3),

-- Ольга Васильевна (ID 4) - есть посылка в 2022
(9.0, 2200, '2022-09-05 10:10:00', 'Delivered', 2, 4, 4),

-- Ольга Васильева (ID 5) - нет посылок вообще (удовлетворяет)
(10.0, 3000, NULL, 'Pending', 3, 4, 5),

-- Иван Иванов (ID 6) - есть посылка в 2022
(2.5, 500, '2022-03-18 15:30:00', 'Delivered', 4, 1, 6),

-- Посылка для физического лица (Мария Иванова)
(3.8, 1800, '2023-04-10 16:20:00', 'Delivered', 6, 2, 7);
```

Запрос с условиями к таблице-предку (SenderCompany) и потомку (LegalSender):

```
SELECT
    CompanyID,
    CompanyName,
    'Основная компания' AS CompanyType
FROM
```

SenderCompany

Query Query History

```sql
SELECT
    CompanyID,
    CompanyName,
    'Основная компания' AS CompanyType
FROM
    SenderCompany
```

Data Output Messages Notifications

	companyid [PK] integer	companyname character varying (30)	companytype text
1	5	МегаДоставка Сервис	Основная компания
2	1	ООО Альфа	Основная компания
3	2	ООО Ромашка	Основная компания
4	3	ООО Гамма	Основная компания
5	4	ООО Бета	Основная компания
6	6	Иванова Мария Сергеевна	Основная компания

Запрос только к таблице-предку (SenderCompany):

```
SELECT
    CompanyID,
    CompanyName,
    'Основная компания' AS CompanyType
FROM
    ONLY SenderCompany
```

Query Query History

```sql
SELECT
    CompanyID,
    CompanyName,
    'Основная компания' AS CompanyType
FROM
    ONLY SenderCompany
```

Data Output Messages Notifications

	companyid [PK] integer	companyname character varying (30)	companytype text
1	5	МегаДоставка Сервис	Основная компания

Запрос только к таблице-потомку (LegalSender):

```
-- Выбираем только юридических лиц с ИНН, содержащим '555'
-- без привязки к посылкам
SELECT
    ls.CompanyID,
    sc.CompanyName,
```

```
    ls.INN
FROM
    LegalSender ls
JOIN
    SenderCompany sc ON ls.CompanyID = sc.CompanyID
WHERE
    ls.INN LIKE '%555%'  -- Условие только к потомку
ORDER BY
    ls.INN;
```

Query Query History

```sql
-- Выбираем только юридических лиц с ИНН, содержащим '555'
-- без привязки к посылкам
SELECT
    ls.CompanyID,
    sc.CompanyName,
    ls.INN
FROM
    LegalSender ls
JOIN
    SenderCompany sc ON ls.CompanyID = sc.CompanyID
WHERE
    ls.INN LIKE '%555%'  -- Условие только к потомку
ORDER BY
    ls.INN;
```

Data Output Messages Notifications

	companyid integer	companyname character varying (30)	inn character varying (12)
1	3	ООО Гамма	555555555555

Придумать и создать пользовательскую агрегатную функцию для своей предметной области:

```
CREATE OR REPLACE FUNCTION compare_addresses(
    a AddressComponents,
```

```
    b AddressComponents
) RETURNS INTEGER AS $$
BEGIN
    -- Возвращает:
    -- -1 если a < b
    -- 0 если a = b
    -- 1 если a > b

    IF a.HouseNumber IS DISTINCT FROM b.HouseNumber THEN
        IF a.HouseNumber < b.HouseNumber OR b.HouseNumber IS NULL
THEN
            RETURN -1;
        ELSE
            RETURN 1;
        END IF;
    END IF;

    IF a.BuildingNumber IS DISTINCT FROM b.BuildingNumber THEN
        IF a.BuildingNumber < b.BuildingNumber OR b.BuildingNumber IS
NULL THEN
            RETURN -1;
        ELSE
            RETURN 1;
        END IF;
    END IF;

    -- Если BuildingNumber равны, сравниваем ApartmentNumber
    IF a.ApartmentNumber < b.ApartmentNumber OR b.ApartmentNumber IS
NULL THEN
        RETURN -1;
```

```
    ELSIF a.ApartmentNumber > b.ApartmentNumber THEN
        RETURN 1;
    ELSE
        RETURN 0;
    END IF;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE OR REPLACE FUNCTION min_address_accum(
    agg_state AddressComponents[],
    new_value AddressComponents
) RETURNS AddressComponents[] AS $$
DECLARE
    comparison INTEGER;
BEGIN
    -- Пропускаем полностью NULL-адреса (если нужно их игнорировать)
    IF new_value = ROW(NULL, NULL, NULL)::AddressComponents THEN
        RETURN agg_state;
    END IF;

    -- Начальное состояние (NULL) заменяем на массив с первым адресом
    IF agg_state IS NULL OR array_length(agg_state, 1) = 0 THEN
        RETURN ARRAY[new_value];
    END IF;

    -- Сравниваем новый адрес с текущим минимальным
    comparison := compare_addresses(new_value, agg_state[1]);

    IF comparison < 0 THEN
```

```
        -- Новый адрес меньше всех предыдущих - заменяем весь массив
        RETURN ARRAY[new_value];
    ELSIF comparison = 0 THEN
        -- Новый адрес равен текущему минимальному - добавляем в массив
        RETURN agg_state || new_value;
    ELSE
        -- Новый адрес больше - оставляем массив без изменений
        RETURN agg_state;
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION min_address_final(
    agg_state AddressComponents[]
) RETURNS AddressComponents[] AS $$
BEGIN
    RETURN agg_state;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE AGGREGATE min_addresses(AddressComponents) (
    SFUNC = min_address_accum,
    STYPE = AddressComponents[],
    FINALFUNC = min_address_final,
    INITCOND = '{}'
);

-- Найдём все минимальные адреса в таблице
SELECT unnest(min_addresses(AddressParts)) AS minimal_addresses
```

FROM DeliveryAddress

WHERE addressid IN (1, 6, 10);

Query Query History

```
1 SELECT unnest(min_addresses(AddressParts)) AS minimal_addresses
2 FROM DeliveryAddress
3
```

Data Output Messages Notifications

	minimal_addresses addresscomponents
1	(10,0,2)

Код операторов:

```
CREATE OR REPLACE FUNCTION can_deliver_by_weight(p_weight DECIMAL, max_weight DECIMAL)
RETURNS BOOLEAN AS $$
BEGIN
   RETURN p_weight <= max_weight;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE OPERATOR <<#>> (
   LEFTARG = DECIMAL,    -- Вес посылки
   RIGHTARG = DECIMAL,   -- Максимальный допустимый вес
   PROCEDURE = can_deliver_by_weight
);

-- Проверка каких посылок можно доставить при максимальном весе 8 кг
SELECT
   p.PackageID,
   p.Weight,
   sc.CompanyName,
   CASE
      WHEN p.Weight <<#>> 8.0 THEN 'Доставляемо'
      ELSE 'Превышение веса'
   END AS delivery_status
FROM
   Package p
JOIN
   SenderCompany sc ON p.SenderID = sc.CompanyID
WHERE
```

p.Weight <<#>> 10.0;

Query Query History

```sql
-- Проверка каких посылок можно доставить при максимальном весе 8 кг
SELECT
    p.PackageID,
    p.Weight,
    sc.CompanyName,
    CASE
        WHEN p.Weight <<#>> 8.0 THEN 'Доставляемо'
        ELSE 'Превышение веса'
    END AS delivery_status
FROM
    Package p
JOIN
    SenderCompany sc ON p.SenderID = sc.CompanyID
WHERE
    p.Weight <<#>> 10.0;
```

Data Output Messages Notifications

	packageid integer	weight numeric (10,2)	companyname character varying (30)	delivery_status text
1	5	4.00	ООО Альфа	Доставляемо
2	4	5.50	ООО Альфа	Доставляемо
3	3	7.00	ООО Альфа	Доставляемо
4	2	6.00	ООО Альфа	Доставляемо
5	1	5.20	ООО Альфа	Доставляемо
6	7	9.00	ООО Ромашка	Превышение веса
7	6	8.00	ООО Ромашка	Доставляемо
8	8	10.00	ООО Гамма	Превышение веса
9	9	2.50	ООО Бета	Доставляемо
10	10	3.80	Иванова Мария Сергеевна	Доставляемо

Вывод:

В ходе работы была создана нормализованная структура базы данных, включающая таблицы-предки и потомки для разделения юридических и физических отправителей, реализованы триггеры для поддержания целостности данных, добавлены пользовательские операторы и агрегатные функции для предметной области. Результатом стала эффективная система учета посылок с корректными связями между сущностями и автоматизированными проверками, соответствующая требованиям предметной области.